

2020 시스템 프로그래밍
- Lab 03. DataLab-

제출일자	2020.10.10
분 반	02
이 름	최재범
학 번	201702081

실습 1

1 bitOr

```
int bitOr(int x, int y) {
    return ~(~x & ~y);
}
```

OR 연산식 전체에 NOT을 씌우면, 각 항이 반전됨과 동시에 OR -> AND로 변함. 따라서 AND를 안써도 됨. 이를 다시 반전시키면 OR 가 됨

2 tmin

```
int tmin(void) {
    int a = 1;
    // 11111111 11111111 11111111 11111111 = -1
    // 10000000 00000000 00000000 00000000 = -2147483648

    return a << 31;
}
```

2의 보수 중 가장 작은 값 : MSB만 1이고 나머지는 다 0, 이는 1
따라서 LSB만 1인 경우를 왼쪽으로 계속 쉬프트해서 MSB까지 보내면 됨
총 32비트 이므로 31번 밀면 됨

3 evenBits

```
int evenBits(void) {
    // 연산자 수를 줄이기 위해, 이전 값을 재활용 : 1->2->4->8바이트
    // 0x 01
    int a = 0x01;

    // 0x 0101
    int b = (a << 2) | a;

    // 0x 01010101
    int c = (b << 4) | b;

    // 0x 01010101 01010101
    int d = (c << 8) | c;

    // 0x 01010101 01010101 01010101 01010101
    int e = (d << 16) | d;

    return e;
}
```

한번에 1바이트씩만 표현 가능하므로, OR연산으로 합쳐줘야 함
하지만 한번에 1바이트씩 OR하면 연산자 수가 모자르게 됨
따라서 이전 값을 재활용하여, 1->2->4->8 바이트로 만들어야 함

4 getByte

```

int getByte(int x, int n) {
    // n = 3 : 8 * 3만큼 우측쉬프트 ~ *는 3번 left shift로 해결
    // n = 2 : 8 * 2만큼 우측쉬프트
    // n = 1 : 8 * 1만큼 우측쉬프트
    // n = 0 : 쉬프트 안함
    // 이후, 맨 오른쪽 바이트만 뽑아냄 : AND with 0xFF
    int shift = n << 3;
    return (x >> shift) & 0xFF;
}

```

1바이트 -> 8비트이므로 3번의 shift로 바이트 단위 이동 가능
 오른쪽부터 위치를 세기 때문에, 우측 쉬프트를 인덱스 바이트 만큼 하면
 LSB로 해당 바이트를 뽑아낼 수 있음 -> 3 * 인덱스 만큼의 쉬프트 필요
 이후 LSB만 뽑아내기 위해 마지막 바이트만 1인 0xFF와 AND함

5	float_abs
---	-----------

```

unsigned float_abs(unsigned uf) {

    unsigned int input = uf;

    // MSB mask용 0111~1
    // 0x80 : 1000 0000
    unsigned int mask_msb = ~(0x80 << 24);

    // Inf : 0xFF800000
    unsigned int inf = ((0xFF << 8) | 0x80) << 16;

    // 특수값 판별을 위해 exp 뽑아냄
    // unsigned int mask_exp = (0xFF) << 23; // 연산자제한
    // unsigned int exp = (input & mask_exp) >> 23;
    unsigned int exp = (input << 1) >> 24;

    // NaN 걸러냄 : exp(8bit) : 0xFF, frac != 0
    if(exp == 0xFF) {
        if(input != inf) return uf; // NaN
    }

    return (input & mask_msb);
}

```

1000 ~ 0 과 AND해서 MSB를 0으로 바꿔주면 되지만, 특수값일 수 있기 때문에 exp인
 부분을 분리해서 확인해 봐야 함. float에서는 exp가 8비트(1바이트), frac이 23비트이기
 때문에 한 바이트만 뽑아서 우측으로 밀면 됨. 하지만 0xFF를 왼쪽으로 밀어서 AND하고
 다시 오른쪽으로 밀면 연산자 제한에 걸리기 때문에, 왼쪽으로 1번 밀어서 MSB만 지우고
 오른쪽으로 23번 + 1번만 밀면 됨 (비부호형이라 0으로 채워짐). 이렇게 뽑아낸 exp값이 다
 1일 경우에는 특수값인데, frac이 0인 경우는 무한대이고 0이 아니면 NaN이 됨. 무한대에도
 부호가 존재하기 때문에 NaN일 경우에만 uf를 그대로 리턴, 아닌 경우에는 MSB만 0으로
 만들어서 리턴

6	addOK
---	-------

```

int addOK(int x, int y) {
    int sum = x + y;
    int mask_msb = 0x1 << 31;

    // MSB 뽑아냄 (unsigned 못써서 산술시프트가 되므로 LSB빼고 다 지움)
    int x_msb = (x & mask_msb) >> 31 & 0x1;
    int y_msb = (y & mask_msb) >> 31 & 0x1;
    int sum_msb = (sum & mask_msb) >> 31 & 0x1;

    // sum에서 msb 변동이 생겼을 시, 오버플로우 발생

    // Maxterm들의 곱으로 결과값 표현 : (x'+y'+s)(x+y+s')
    return (!x_msb | !y_msb | sum_msb) & (x_msb | y_msb | !sum_msb);
}

```

x	y	sum	overflow	return
0	0	0	0	1
0	0	1	1	0
0	1	0	0	1
0	1	1	0	1
1	0	0	0	1
1	0	1	0	1
1	1	0	1	0
1	1	1	0	1

두 정수와 그 합의 msb를 각각 구한 후 진리표를 그려서 식으로 나타냄. msb를 구하기 위해 msb 비트만 and하고 lsb까지 우측으로 미는데, signed라서 그냥 밀면 msb가 계속 1로 채워지므로 0x1과 and해서 다른 비트를 다 지워줘야 함. overflow가 발생하면 0을 리턴하므로, 이를 식으로 나타내면 $(x'+y'+s)(x+y+s')$ 가 되고 이것을 리턴함.

7 replaceByte

```

int replaceByte(int x, int n, int c) {

    // n번 위치의 바이트를 0으로 만들고, 대상 값으로 OR mask
    // n << 3 : 8*n
    int erase_mask = ~(0xFF << (n << 3));
    int obj_mask = c << (n << 3);

    return (x & erase_mask) | obj_mask;
}

```

n번째 위치의 바이트만 0과 and하여 0으로 만들어줌. mask를 만들때, 1을 계속 shift하고 or하는 것 보다, 해당 위치만 1로 만들어서 NOT하는 것이 연산자를 덜 쓸 수 있음. 바이트 단위 n번 좌측이동을 3n번의 비트 좌측시프트로 표현하여, 1로 채워진 바이트를 원하는 위치에 넣어주고 이를 NOT한 뒤에 x와 AND하여 해당 바이트를 지워줌. 이후 대체할 c값도 3n번 좌측시프트하여 n번째 바이트에 넣어준 뒤에 or하여 덮어 씌움.

8 isNonZero

```

int isNonZero(int x) {

    // 0은 2의 보수를 취해도 똑같음
    // 보수를 취할 때 여러 자리에서 올림이 발생 가능, MSB가 올림되는 경우는 0밖에 없음
    // 0이 아닐 때 : 2의 보수를 취할 때, 전과 후 어느 한 쪽은 반드시 msb가 1
    // msb 기준으로 구분 (msb == 0이면 0, msb == 1이면 1)
    return ((x | (~x + 1)) >> 31) & 0x1;

}

```

리턴값이 0이거나 1이어야 하는데, 입력값은 다른 정수가 올 수 있음. 따라서 0과 0이 아닌 것의 차이를 이용하여 논리연산을 해야 하는데, 0은 2의 보수를 취해도 전부 0으로 아무 비트도 변하지 않지만 다른 정수는 MSB가 전과 후 중 하나는 무조건 1을 가지게 됨. 따라서 0이 아닌 정수와 그것에 2의 보수를 취한 값의 msb를 OR하면 1이 되므로, 0과 구별 가능.